▶ 1분 미리보기

Chapter 2. Bedrock 위에서 안전하게 쓰기

키는 나눠 주지 않고, 정책은 파일로, 막을 것은 앞에서 (Claude Code v2.1.319, Bedrock 기준)

Oh, Junseok

Sr. Solutions Architect

AWS Korea

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

열에 여섯이 고른 주제, 안전하게 쓰기

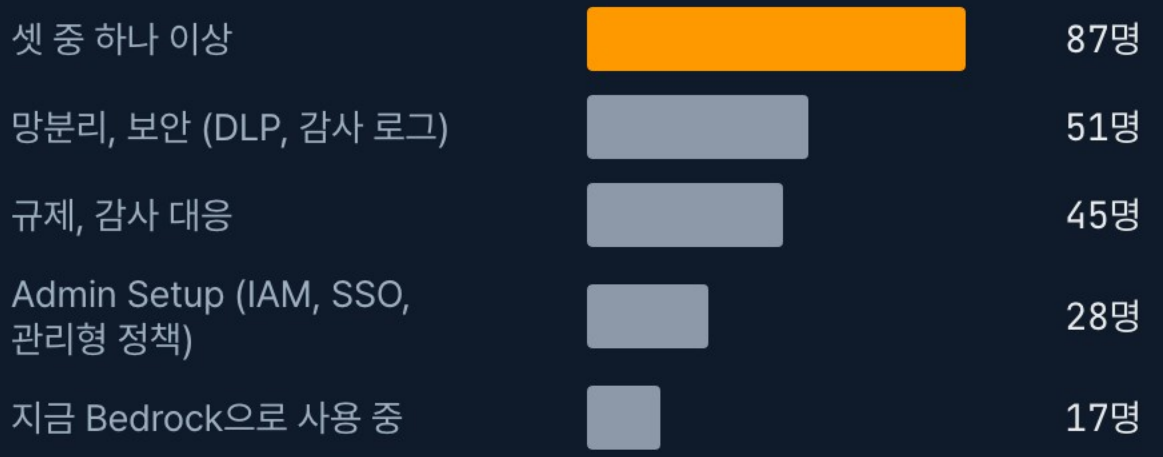
사전 설문 결과로 정한 이 챕터의 목표

PRE-SURVEY

87명

참가 확정자 **142명 중 87명**이 망분리와 보안, 규제와 감사 대응, Admin Setup 가운데 하나 이상을 기대 주제로 골랐습니다. 모두 이 챕터가 답하는 주제입니다.

그런데 지금 Bedrock으로 쓰는 분은 17명입니다. 그래서 Bedrock에 닿는 길부터 그림으로 보고, 정책 배포와 가드레일로 넘어갑니다.



기대 주제는 복수 선택, 막대 길이는 응답 인원

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

핵심은 강의, 깊이는 실습

세 개의 Part, 연결에서 정책과 가드레일까지

PART 01

Bedrock에 연결하기

사내망에서 Bedrock까지의 길, 환경변수와 IAM, 키 대신 SSO, 모델 고정과 팀 이름표

PART 02

정책은 파일로

파일 한 장이 모든 PC로, 키 충돌과 잠금 스위치, 배포 후 확인

PART 03

무엇을 못 하게 할 것인가

deny가 이긴다, 요청이 지나는 문들, DLP는 프롬프트 앞에서, 남는 기록

LAB / Ch2 Lab 1

정책 배포, managed settings와 게이트웨이 그룹 정책

정책을 배포하고, 개인 설정으로 우회가 안 되는지 확인

LAB / Ch2 Lab 2

가드레일과 DLP, 훅과 auto mode

훅을 만들고 직접 뚫어 본 뒤 고치기

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

금융권이 Bedrock을 고르는 이유

대신 claude.ai의 관리 콘솔, Analytics, 정책 서버가 없음

01

계정 경계 안의 추론

처리 리전은 추론 프로필 범위를 따름 (global은 리전 밖 처리 가능)

02

기존 통제의 재사용

IAM, IAM Identity Center, CloudTrail, Organizations

03

청구 일원화

AWS 청구서와 Cost Explorer, Budgets로 관리

04

빈자리는 직접 채움

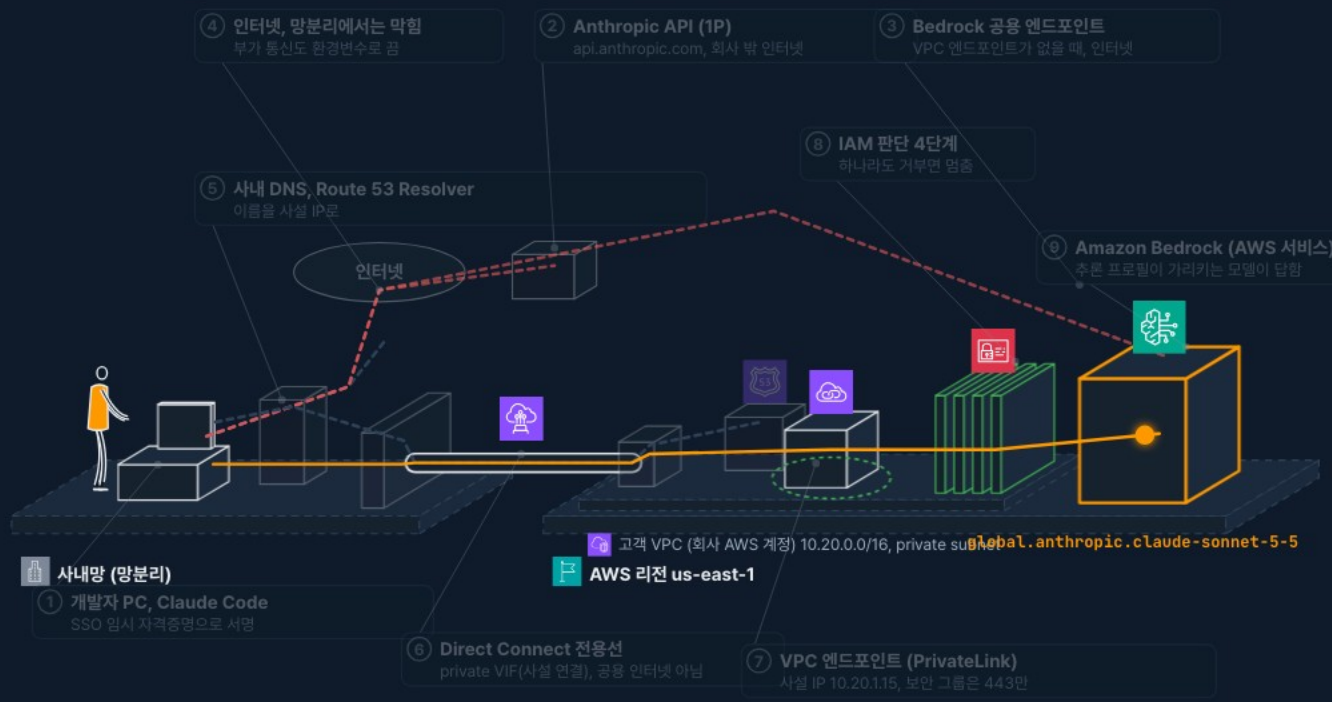
정책은 파일과 MDM으로, 관측은 OTel(OpenTelemetry)로

claude.ai의 관리 콘솔, Analytics, 정책 서버가 없는 자리를 무엇으로 채우는지, 그것이 이 챕터와 Ch6의 이야기입니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

사내망에서 Bedrock까지, 인터넷 없이

요청 하나가 지나가는 길 (사내망과 전용선 기준)



10 응답

답은 같은 길로 돌아온다

```
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=us-east-1
export AWS_PROFILE=fsi-dev   # aws sso login
export CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1
export ANTHROPIC_MODEL=global.anthropic.claude-
sonnet-5-5
# DNS: bedrock-runtime ... → 10.20.1.15
# IAM: bedrock:InvokeModel*
```

- 서명 확인 (SigV4: AWS 요청 서명)
- 역할 정책에 bedrock:InvokeModel*
- 엔드포인트 정책이 허용
- 조직 SCP(계정 전체 권한 상한)가 막지 않음

답도 같은 길로 돌아옵니다. **처음부터 끝까지 공용 인터넷을 지나지 않습니다.** 2, 3단계의 길과 비교해 보세요.

장면을 누르거나 ↓ 키로 다음 단계, ↑ 키로 이전 단계

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

망분리 환경의 여덟 가지 점검

인터넷 없이 Bedrock만 열려 있을 때 먼저 확인할 것

모델 호출

Bedrock Runtime VPC 엔드포인트

PrivateLink로 계정 안에서 추론 (앞 장의 길)

SSO 로그인

Identity Center, OIDC, STS

aws sso login이 되는지 먼저 확인 (OIDC: 로그인(인증) 표준, STS: 임시 자격증명 발급)

사내 프록시

HTTPS_PROXY, NO_PROXY

프록시를 거쳐야 하는 경우

사내 인증서

NODE_EXTRA_CA_CERTS

프록시가 TLS(HTTPS 암호화)를 다시 서명할 때

외부 통신 끄기

CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1

자동 업데이트, 오류 리포트 등을 한 번에

설치와 업데이트

사내 배포, DISABLE_AUTOUPDATER=1

버전을 정해 단말에 배포

플러그인

사내 git 마켓플레이스

Ch4에서 실습

웹 접근

WebFetch deny 또는 허용 도메인

허용 도메인은 skipWebFetchPreflight: true와 함께, WebSearch는 Bedrock에 없음

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 02 / PART 01

Bedrock에 연결하기

첫 세션까지 가장 짧은 길

01
환경변수와 IAM 권한

02
키 대신 SSO 로그인

03
모델 고정과 팀 이름표

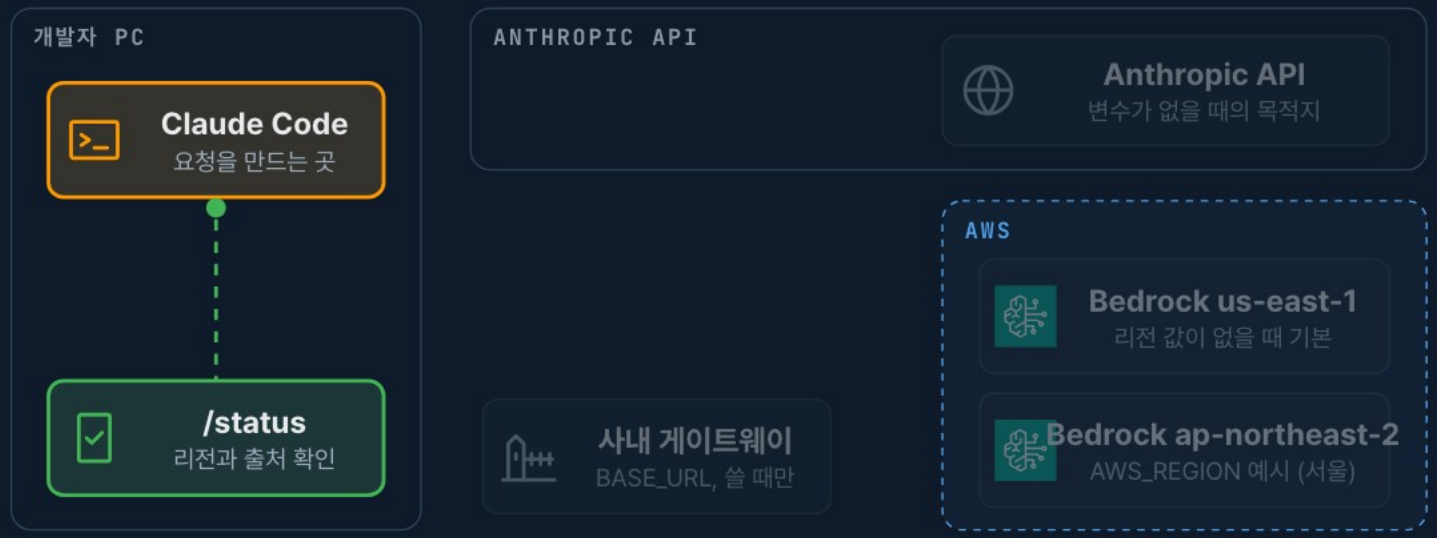
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

환경변수 두 개로 연결

변수마다 바뀌는 것 하나: 목적지, 리전, 경유지

STEP 4 /status로 확인

설정이 실제로 먹었는지는 Claude Code 안에서 **/status**로 봅니다. 리전이 설정 파일이나 기본값에서 왔다면 **그 출처**도 함께 나옵니다.



● Bedrock 연결 ● Bedrock 리전 ● 경유지 (선택) ● 확인

확인

Claude Code 프롬프트

```
/status
```

Lab에서도 연결 직후 **/status**부터 봅니다.

- 공급자가 Amazon Bedrock인지
- 리전과 그 출처

‹ 5 / 5 ›

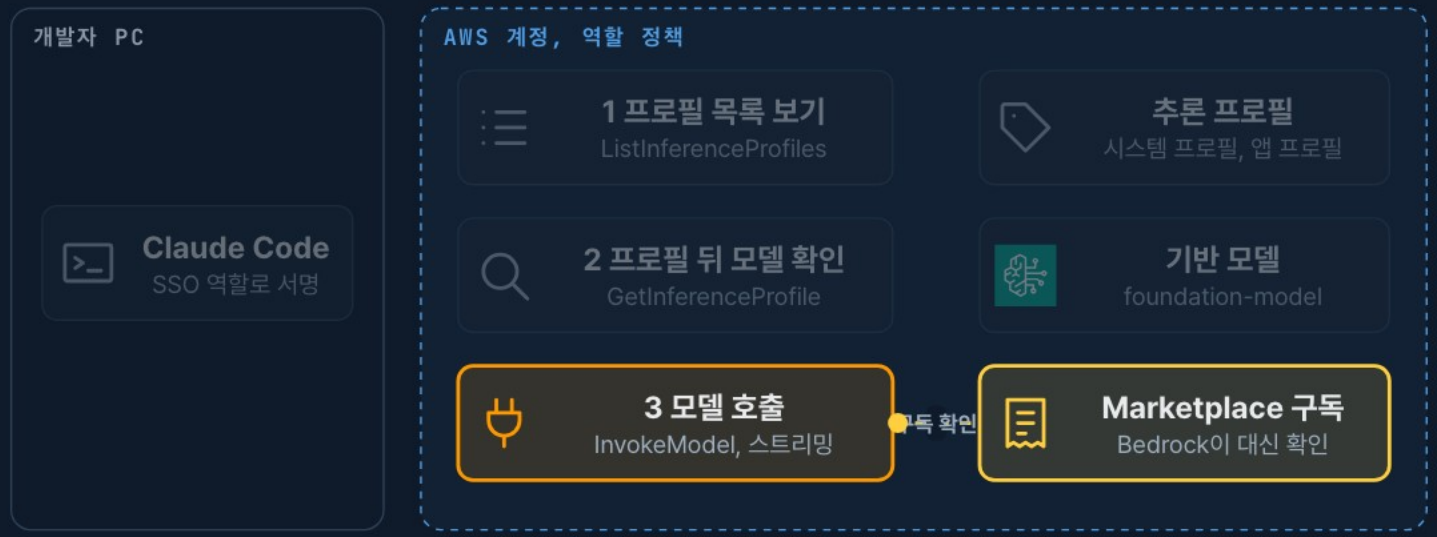
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

IAM 권한은 호출, 조회, Marketplace

Claude Code가 부르는 API마다 필요한 권한

STEP 4 **Marketplace: Bedrock을 거칠 때만**

Anthropic 모델은 Marketplace 구독으로 제공되어, 호출할 때 Bedrock이 **구독을 확인**합니다. 조건 **aws:CalledViaLast**로 Bedrock을 거친 요청에만 허용합니다.



● 모델 호출 ● 프로필 조회 ● Marketplace

문장 3

claude-code-bedrock-policy.json

```
{ "Effect": "Allow",
  "Action": [ "aws-
marketplace:ViewSubscriptions",
              "aws-marketplace:Subscribe"
],
  "Resource": "*",
  "Condition": { "StringEquals": {
    "aws:CalledViaLast":
"bedrock.amazonaws.com" } } }
```

모델 사용 신청(use case 양식)은 계정마다 한 번, 콘솔에서 합니다.

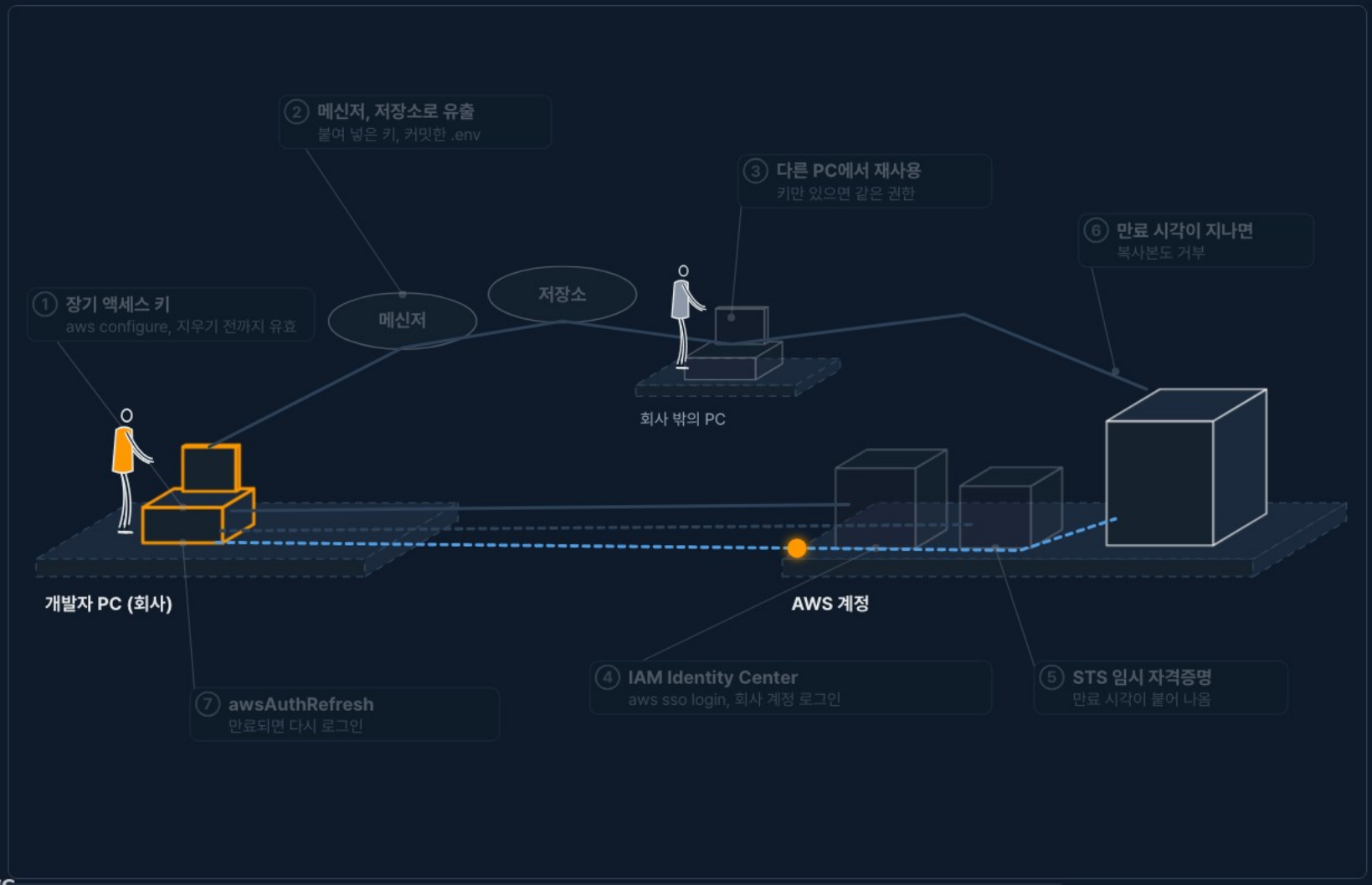
- ViewSubscriptions, Subscribe
- 개발자가 직접 구독하는 권한이 아님

‹ 5 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

복사되는 키, 만료되는 SSO

키는 나눠 주지 않고, 로그인해서 짧게 빌려 쓰기



8 이어서 호출

새 자격증명으로 같은 요청을 다시

```
# 나눠 주는 키 (비권장)
aws_access_key_id = AKIA…EXAMPLE
# SSO 프로필 (권장)
aws sso login --profile fsi-dev
export AWS_PROFILE=fsi-dev
"awsAuthRefresh":
  "aws sso login --profile fsi-dev"
```

로그인이 끝나면 새 자격증명으로 **요청을 다시 보내고**, 세션은 그대로 이어집니다.

장면을 누르거나 ↓ 키로 다음 단계, ↑ 키로 이전 단계

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

다섯 가지 중 권장은 하나

Bedrock API 키만 AWS SDK 기본 체인을 거치지 않음

STEP 4 **Bedrock API 키: 체인 밖**

AWS_BEARER_TOKEN_BEDROCK이 있으면 Claude Code는 다른 자격증명이 있어도 **체인을 보지 않고** 이 키로만 인증합니다.



● 권장 ● 보조 ● 장기 키 ● 체인 밖

Bedrock API 키

Terminal

```
export AWS_BEARER_TOKEN_BEDROCK=...
# 체인과 자격증명 캐시를 거치지 않음
```

키 하나가 곧 권한이라, **나눠 주는 키**와 같은 문제가 생깁니다.

- 단기 키: 최대 12시간
- 장기 키: AWS도 탐색용으로만 권장
- 배포용으로는 비권장

‹ 5 / 5 ›

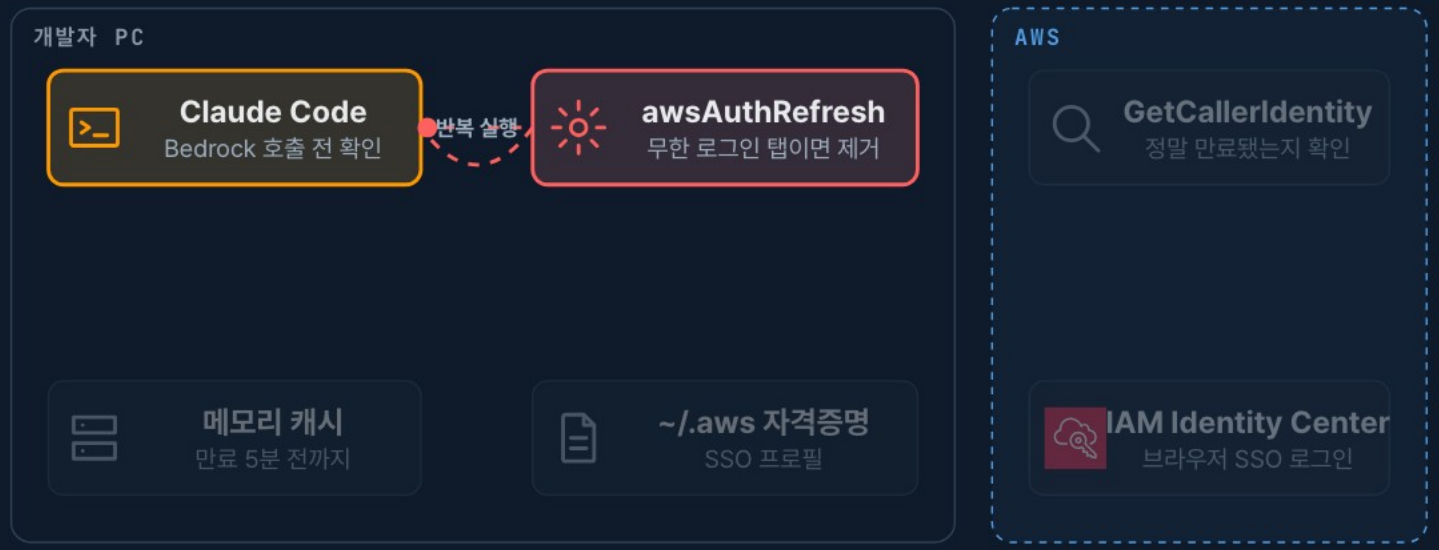
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

만료 시 자동 재로그인

SSO 자동 갱신, awsAuthRefresh와 awsCredentialExport

STEP 4 **막힐 때 보는 곳**

VPN이나 TLS 검사 프록시가 브라우저 로그인을 끊으면 로그인 탭이 **계속 열립니다**.
이때는 **awsAuthRefresh를 빼고** 시작 전에 aws sso login을 직접 실행합니다.



Identity Center와 함께

managed-settings.json (예시)

```
{
  "awsAuthRefresh": "aws sso login --
profile fsi-dev",
  "env": {
    "CLAUDE_CODE_USE_BEDROCK": "1",
    "AWS_PROFILE": "fsi-dev",
    "AWS_REGION": "us-east-1"
  }
}
```

Identity Center **권한 세트로 Bedrock 최소 권한**을 줍니다.

- 확인 호출도 HTTPS_PROXY를 따름
- 사내 루트 인증서: OS 저장소나 NODE_EXTRA_CA_CERTS
- 무한 로그인 탭: awsAuthRefresh 제거

● Claude Code ● 자격증명 흐름 ● 갱신 명령 ● 문제 상황

‹ 5 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

고정하지 않으면 무엇이 도는가?

QUICK CHECK, 손을 들어 답해 주세요

모델을 고정하지 않은 Bedrock 배포에서 기본 주 모델은?

A Sonnet 4.5

B Opus 5.5

C Haiku 5.5

D Sonnet 5.5

정답 B. 고정하지 않으면 Opus 단가로 청구됩니다. (amazon-bedrock 문서)

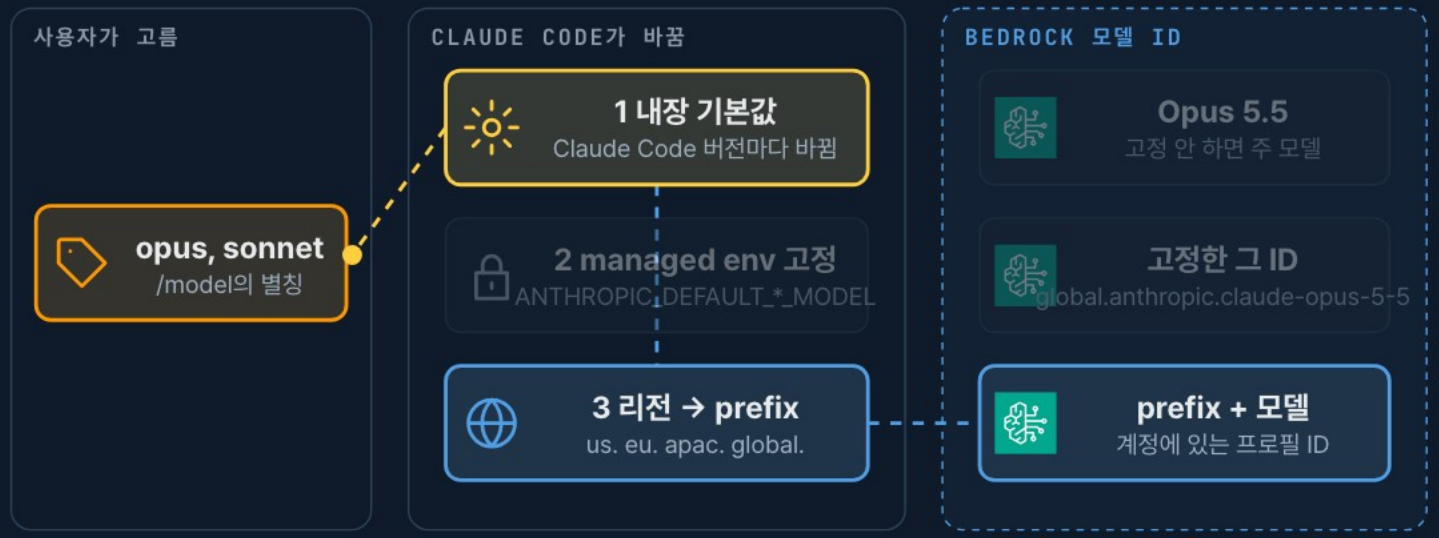
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

여러 사용자에게 배포하면 반드시 고정

별칭이 어느 모델 ID로 가는지 정하기

STEP 4 **prefix만 바꾸는 방법**

ANTHROPIC_BEDROCK_REGION_PREFIX는 내장 기본값은 그대로 두고 prefix만 바꿉니다. 버전까지 정하려면 STEP 2의 고정을 씁니다.



prefix만

환경변수

```
export
ANTHROPIC_BEDROCK_REGION_PREFIX=global
# opus → global.anthropic.
#        claude-opus-5-5
```

예를 들어 us-* 리전에서도 **global.** 프로필을 쓰게 합니다.

- 허용값: us, eu, apac, jp, au, global
- 기본값의 버전은 여전히 바뀔 수 있음

● 별칭 ● 내장 기본값 ● 고정 ● prefix

‹ 5 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

팀마다 다른 이름표, 추론 프로필

누구의 호출이 어느 프로필을 지났는지 남기기

STEP 5 **실제 문자열, 프로필 ARN**

이름표를 만드는 명령과 그 결과입니다. 결과로 나온 **ARN**이 앞 단계의 정책 자리에 들어갑니다.



● 팀 ● 이름표 ● 기록 ● 통제 ● 거부

이름표 만들기

Terminal

```
aws bedrock create-inference-profile \
  --inference-profile-name sonnet-team-a \
  --model-source copyFrom=arn:aws:bedrock:\
us-east-1:111122223333:inference-profile/\
global.anthropic.claude-sonnet-5-5 \
  --tags key=team,value=team-a
# 결과
arn:aws:bedrock:us-east-1:111122223333:
application-inference-profile/a1b2c3d4e5f6
```

ARN은 화면 폭 때문에 나눠 적었습니다. 실제로는 한 줄입니다.

- arn:aws:bedrock: 서비스
- us-east-1: 리전
- 111122223333: 계정 (예시)
- application-inference-profile/: 이름표

‹ 6 / 6 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 02 / PART 02

정책은 파일로

콘솔이 없으니 파일이 정책이다

01
파일 한 장이 모든 PC로

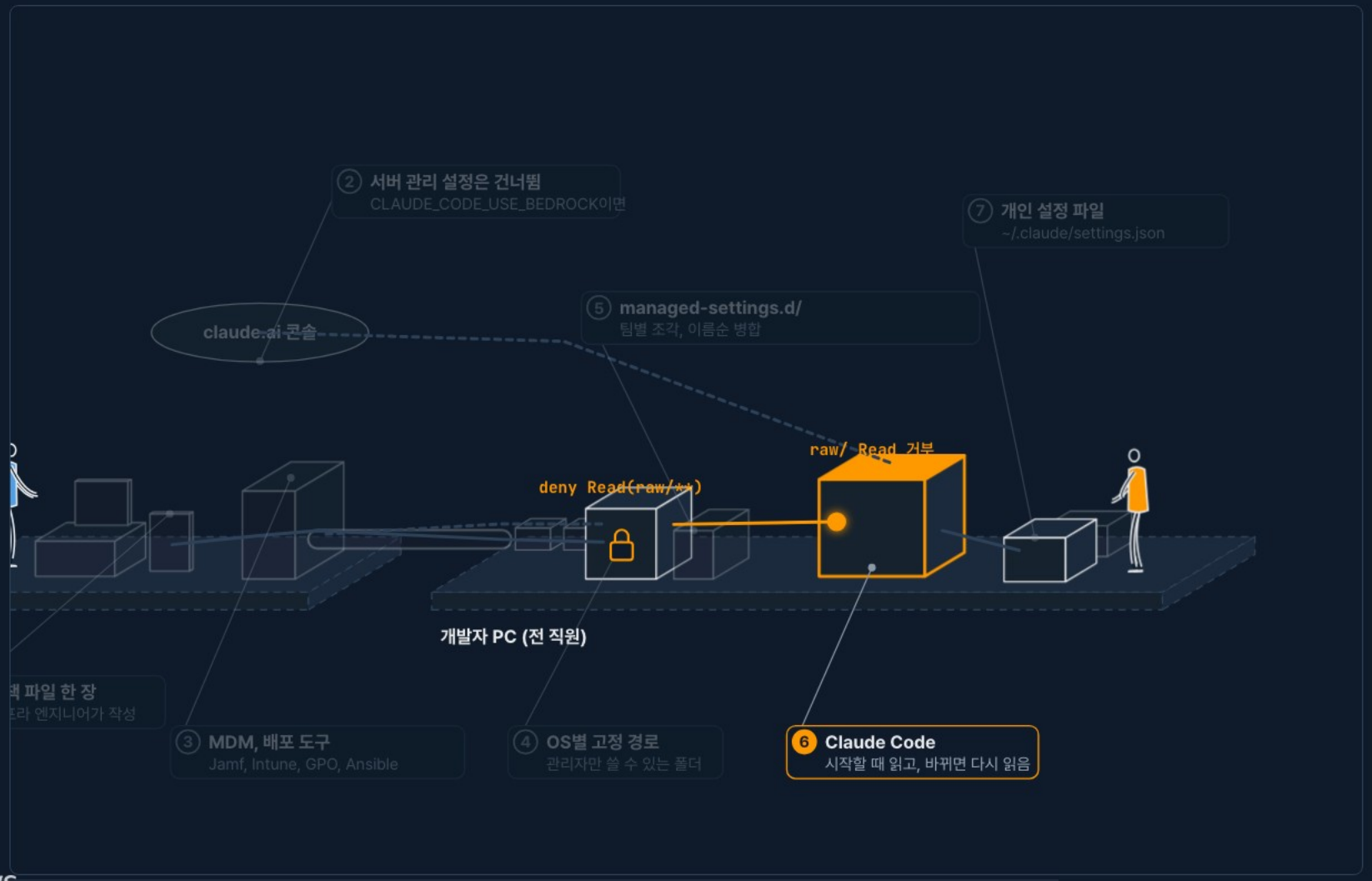
02
키 충돌과 잠금 스위치

03
배포 후 확인

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

콘솔 대신 파일과 MDM

정책 파일 한 장이 모든 개발자 PC로 가는 길



7 결과

managed 규칙이 그대로 이긴다

```
# 인프라 팀 (Linux, WSL 경로)
/etc/claude-code/managed-settings.json
"deny": ["Read(~/claude-lab/ch2/raw/**)"]
# mac /Library/Application
Support/ClaudeCode/
# win C:\Program Files\ClaudeCode\
# 개발자 ~/.claude/settings.json
"allow": ["Read(~/claude-lab/ch2/raw/**)"]
# 결과: raw/ Read는 여전히 거부
```

- 시작할 때 읽고, 바뀌면 다시 읽음
- 사용자, 프로젝트, --settings보다 위
- /status에 Enterprise managed settings

규칙은 모든 층에서 합쳐지고 **deny가 먼저** 평가됩니다. raw/ 읽기는 여전히 거부. 개인이 정책을 되돌릴 수 없습니다.

장면을 누르거나 ↓ 키로 다음 단계, ↑ 키로 이전 단계

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

키 충돌 시 상위 우선

같은 키가 다섯 층에 있을 때 이기는 값

LISTS **목록 키는 고르지 않고 합칩니다**

permissions.deny 같은 목록은 **모든 층의 항목을 합칩니다**. User가 빈 목록을 적어도 조직이 넣은 deny는 남습니다.



목록 키

`permissions.deny`

```
# 1 Managed
"deny": ["Read(~/claude-lab/ch2/raw/**)"]
# 4 Project
"deny": ["Read(./.env)"]
# 5 User
"deny": []

# 이번 세션의 deny (합집합)
Read(~/claude-lab/ch2/raw/**)
Read(./.env)
```

그래서 개인이 **조직의 deny를 지울 수 없습니다**.

- 목록: 모든 층의 항목을 합침
- 예외: managed의 availableModels는 합치지 않고 그대로
- fallbackModel처럼 순서가 있는 목록은 한 층 값만

● Managed (사용자가 못 바꿈) ● 개인과 팀 설정 ● 이기는 값 ● 쓰이지 않는 값

‹ 6 / 6 ›

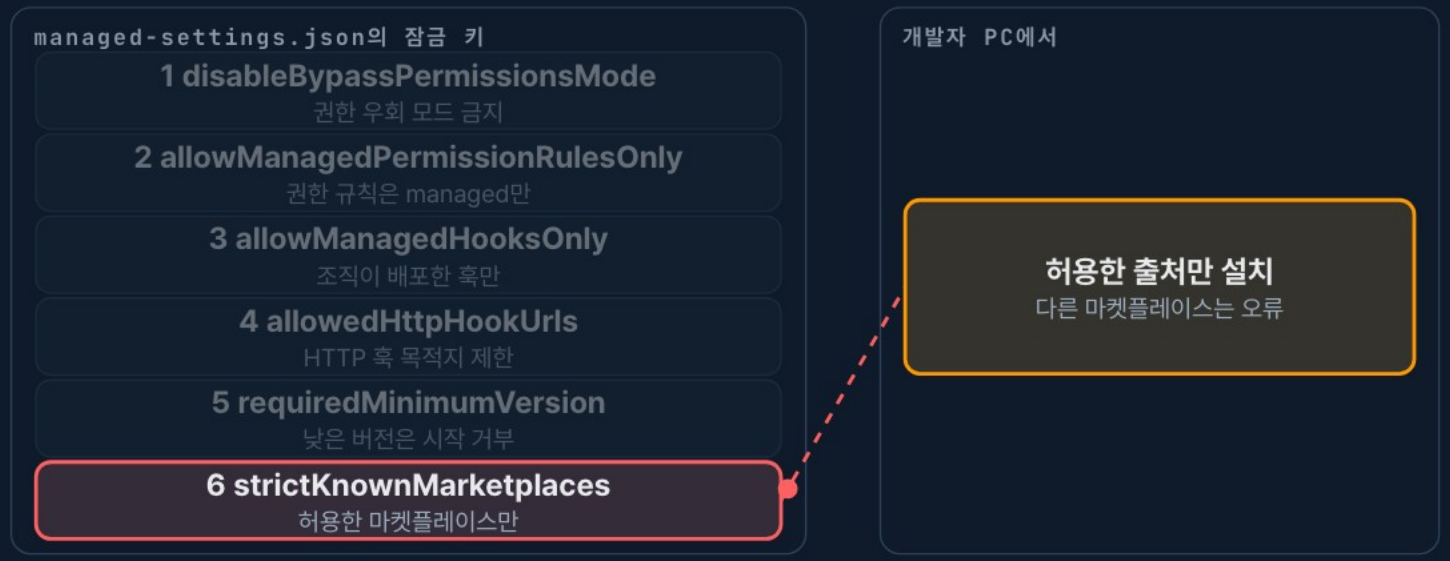
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

조직이 거는 잠금 스위치 여섯 개

managed-settings.json에 넣는 잠금 키와 그 효과

KEY 6 **마켓플레이스는 허용 목록만**

마켓플레이스 추가와 플러그인 설치, 업데이트를 **허용한 출처**로만 제한합니다.



잠금 키 묶음 (예시 값)

/etc/claude-code/managed-settings.json

```
{
  "permissions": {
    "disableBypassPermissionsMode": "disable" },
  "allowManagedPermissionRulesOnly": true,
  "allowManagedHooksOnly": true,
  "allowedHttpHookUrls":
    ["https://audit.corp.example/*"],
  "requiredMinimumVersion": "2.1.300",
  "strictKnownMarketplaces": [{ "source": "git",
    "url": "https://git.corp.example/p.git" }]
}
```

git 주소는 예시입니다.

- 범위: managed 전용
- 빈 배열이면 공식 마켓플레이스까지 차단
- Ch4 확장하기에서 다시 봄

● managed 잠금 키 ● 개발자 PC의 효과

‹ 7 / 7 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab에서 배포할 정책 한 장

managed-settings.json, 줄마다 담은 뜻

BYPASS **우회 모드는 잠급니다**

disableBypassPermissionsMode를 두면 --dangerously-skip-permissions를 줘도 우회 모드로 들어가지 않습니다. deny 규칙도 그대로 살아 있습니다.



Ch2 Lab 1 TASK 02

`/etc/claude-code/managed-settings.json`

```
{
  "availableModels": ["sonnet", "haiku"],
  "enforceAvailableModels": true,
  "env": { "ANTHROPIC_MODEL": "sonnet" },
  "cleanupPeriodDays": 14,
  "permissions": { "deny": [
    "Read(~/claude-lab/mydata/raw/**)",
    "Read(~/claude-lab/ch2/raw/**)" ],
    "disableBypassPermissionsMode":
      "disable" }
}
```

랩 TASK 03에서 **직접 확인**합니다.

- 실행은 exit=0으로 끝남
- 우회 모드만 꺼짐

● 모델 ● 보존 기간 ● 원본 데이터 ● 우회 모드

‹ 5 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

JSON이 깨지면 어떻게 될까?

QUICK CHECK, 손을 들어 답해 주세요

managed-settings.json에 JSON 문법 오류가 있으면?

A 무시하고 실행

B 틀린 줄만 빼고 실행

C 시작을 거부

D 기본값으로 실행

정답 C. "Managed settings document could not be parsed" 오류를 내고 시작하지 않습니다(claude -p는 exit=1). JSON은 맞고 항목 하나만 틀렸다면 그 항목만 빠지고 나머지는 적용됩니다.

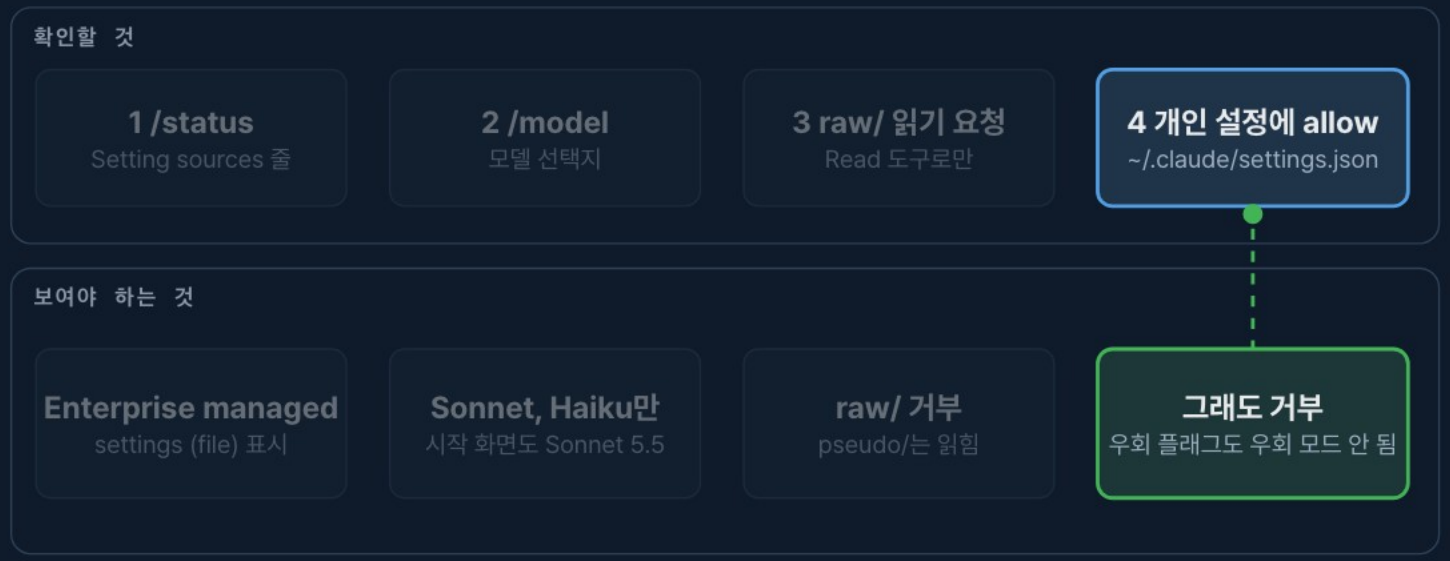
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

배포 이후가 진짜 시작

적용 검증 4단계, 랩 순서 그대로

STEP 4 **개인이 되돌릴 수 없나**

개인 **settings.json**에 allow를 넣어도 읽기는 여전히 거부됩니다. 우회 플래그를 줘도 우회 모드로 들어가지 않습니다.



● 확인할 것 ● 보여야 하는 것

Terminal

~/claude-lab/ch2

```
# ~/.claude/settings.json에 allow 추가
"allow": ["Read(~/claude-lab/ch2/raw/**)"]
claude -p "Use the Read tool to read raw/customer
s.csv"
# → 여전히 거부
claude --dangerously-skip-permissions -p "hi"
# → exit=0, 우회 모드는 꺼진 채
```

이 확인이 **정책이 강제된다**는 증거입니다.

- 권한 규칙은 합쳐지고 deny가 먼저
- 끝나면 개인 설정을 원복

‹ 5 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 02 / PART 03

무엇을 못 하게 할 것인가

앞에서 막고, 놓친 것은 뒤에서, 남는 것은 기록으로

01
권한 규칙과 분류기, 두 개의 문

02
층마다 잡는 것, 프롬프트 앞의 DLP 훅

03
정책으로 강제, 남는 기록

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

deny 절대 우선

권한 평가 순서와 Tool(specifier) 규칙 문법

STEP 5 **4 일치 없음: 모드가 정합니다**

어느 규칙에도 일치하지 않으면 **현재 권한 모드의 기본 동작**을 따릅니다. auto mode라면 여기서 **분류기**가 판정합니다.



현장에서 걸리는 지점

```
# Bash 패턴은 우회가 쉬움: 샌드박스와 함께
# 경로는 ./ 기준 상대 경로
# MCP는 mcp__서버명 단위
# 복잡한 판단은 훅으로
```

사용자가 이 규칙을 못 바꾸게 하는 건 **Part 2의 잠금 키**입니다.

- Manual이면 확인, acceptEdits면 편집 자동 등
- auto mode면 분류기가 판정 (다음 장)
- 복잡한 판단은 훅 (이 Part 뒤쪽)

● 거부 ● 사람 확인 ● 바로 실행 ● 다음 단계로

‹ 6 / 6 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

user allow, managed deny라면?

QUICK CHECK, 손을 들어 답해 주세요

user settings에 allow "Bash(curl *)", managed에 deny "Bash(curl *)". 결과는?

A 그대로 실행

B 확인 후 실행

C 거부

D 나중 설정이 이김

정답 C. 권한 규칙은 합쳐지고, deny가 먼저 평가됩니다.

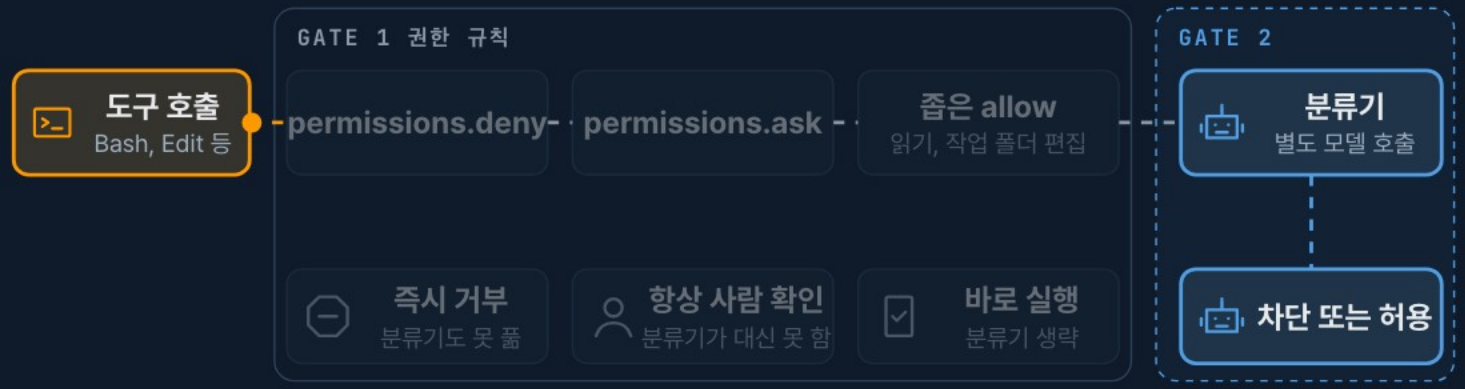
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

권한 규칙이 첫 문, 분류기가 둘째 문

auto mode의 판정 순서

GATE 2 **남은 것은 분류기가 판정합니다**

앞의 문을 모두 지난 호출은 **분류기**가 판정합니다. 순서는 **hard_deny, soft_deny, allow 예외, 사용자 명시 의도**입니다.



분류기 판정 순서

GATE 2

```
hard_deny              # 무조건
soft_deny              # 기본 차단
allow 예외           # soft_deny를 풂
사용자 명시 의도    # 정확히 그 동작을 지시
```

분류기는 **별도 모델 호출**이라 Bedrock에서는 토큰이 듭니다.

- hard_deny: 무조건 막음
- soft_deny: 막되 allow 예외나 명시 의도로 풀림
- "정리해 줘"는 명시 의도 아님, "force push 해 줘"는 명시 의도

● 거부 ● 사람 확인 ● 바로 실행 ● 분류기 판정

‹ 5 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

설정하지 않아도 막는 네 가지

분류기의 기본 규칙이 막는 것과 통과시키는 것

PASS **묻지 않고 통과하는 것**

작업 폴더 파일 작업, lock 파일에 선언된 의존성 설치, 읽기 전용 HTTP는 통과합니다.
현재 저장소로의 push도 기본 통과라서, 꼭 묻게 하려면 ask를 겁니다.



● soft_deny ● hard_deny ● 통과 ● 분류기

기본 규칙 이름 (일부)

`claude auto-mode defaults`

```
$ claude auto-mode defaults
soft_deny:
  Code from External      # curl | bash
  Git Destructive         # force push
  Production Deploy       # 운영 배포
hard_deny:
  Data Exfiltration       # 민감 데이터 반출
allow:
  작업 폴더 파일, lock 의존성, 읽기 HTTP
# "$defaults"를 빼고 쓰면 그 목록이 통째로 사라짐
```

규칙을 더할 때는 **"$defaults"**를 남겨 기본 규칙을 지킵니다.

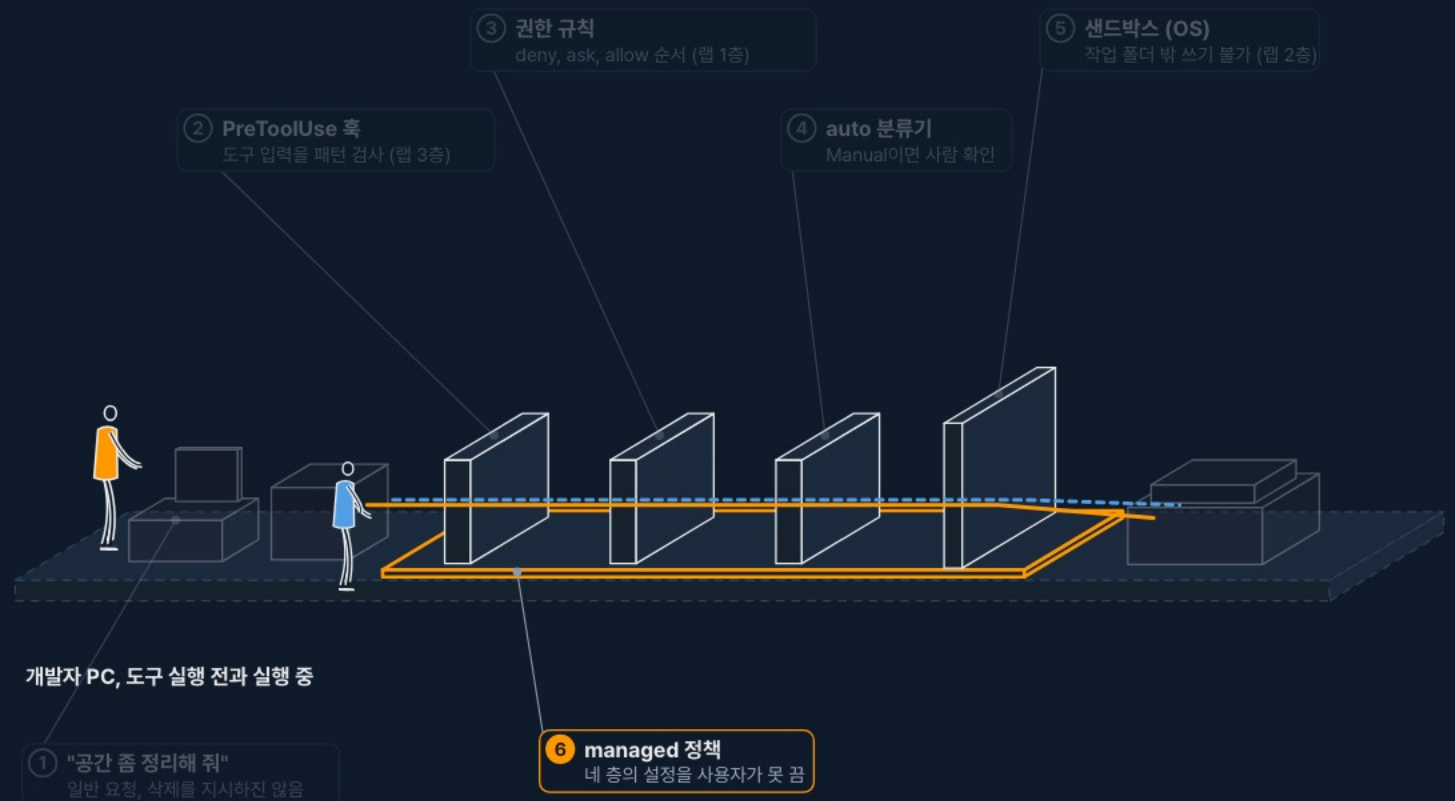
- 작업 폴더 파일, lock 의존성, 읽기 HTTP
- 현재 저장소 push는 기본 통과 (기본 브랜치 포함)
- push만은 묻게: permissions.ask

‹ 6 / 6 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

앞 층이 놓친 것은 다음 층에서

위험한 명령 하나가 지나가는 검사 순서



6 managed 정책

네 층을 정책 파일이 잠근다

```
요청: "공간 좀 정리해 줘"
Bash: rm -rf ../shared-ledger
PreToolUse dlp-guard.sh   → exit 0
"deny": ["Bash(rm -rf *)"] → 거부
Bash: find ../shared-ledger -delete
auto 분류기: soft_deny     → 차단
"sandbox": { "enabled": true } → 쓰기 불가
/etc/claude-code/managed-settings.json
```

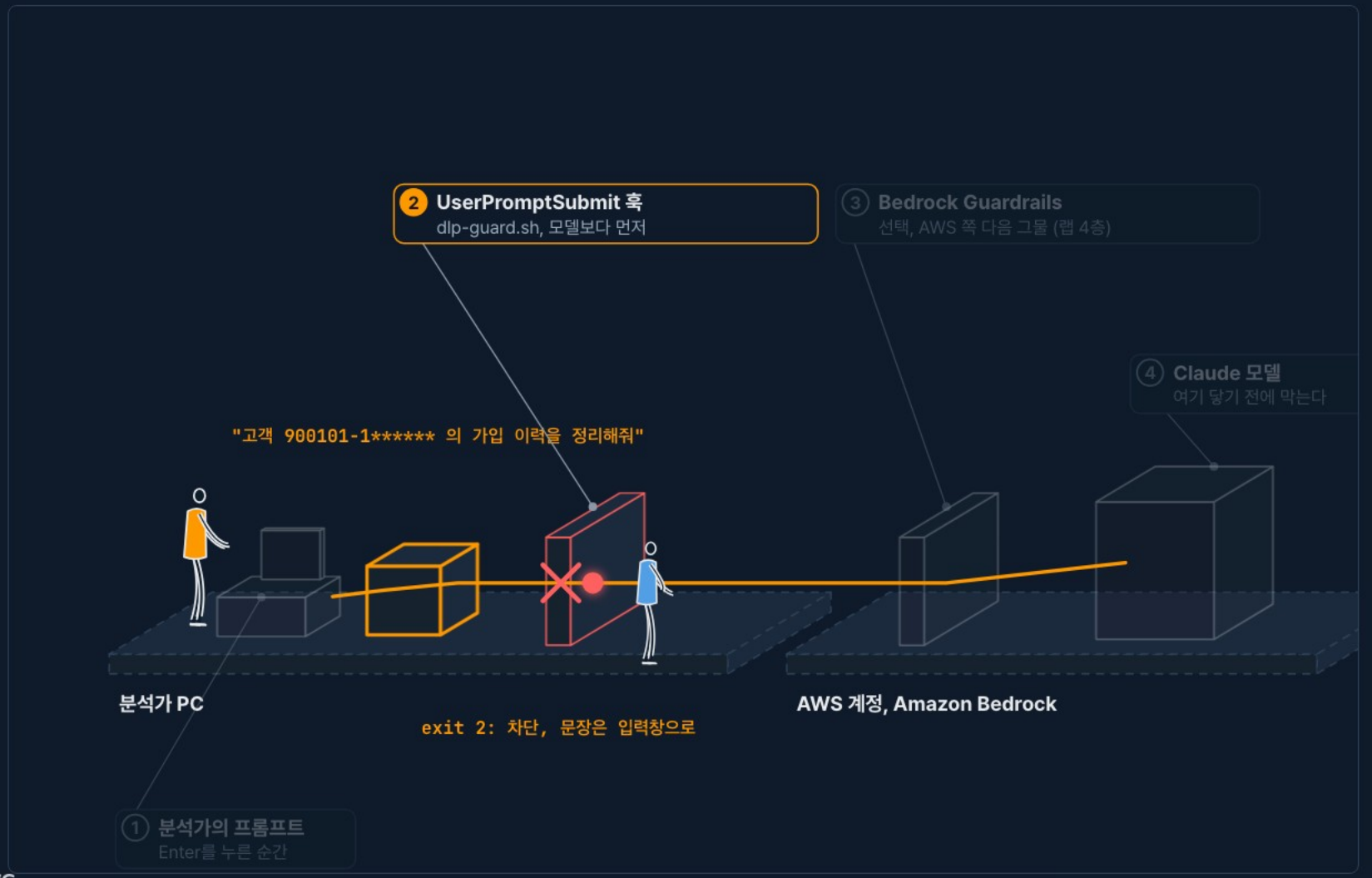
훅, deny, auto 설정, 샌드박스를 **managed-settings.json**에 두면 사용자가 끄거나 덮어쓸 수 없습니다.

장면을 누르거나 ↓ 키로 다음 단계, ↑ 키로 이전 단계

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

DLP는 프롬프트 앞에서

UserPromptSubmit 훅이 모델보다 먼저 본다



4 fail-closed

오류 경로도 exit 2로 막는다

```
"UserPromptSubmit": [{ "hooks": [{
  "command": "/opt/dlp/dlp-guard.sh",
  "onFailure": "block" }] }]  # 2.1.295+
PAT='...[0-9]{6}-[1-4][0-9]{6}...'
일치      → exit 2  # 차단, 사유는 사용자에게
불일치    → exit 0  # Bedrock으로
jq 없음   → exit 0, 127  # 비차단, 통과
command -v jq || exit 2  # 수정본
```

수정본은 jq가 없으면 **exit 2**입니다. 스크립트가 아예 없거나 시간이 초과돼도 **onFailure: block**이 막습니다.

장면을 누르거나 ↓ 키로 다음 단계, ↑ 키로 이전 단계

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

차단은 exit 2뿐

DLP 훅 예제와 fail-open, 그리고 수정본

CASE 4 **수정본은 오류 경로도 막습니다**

수정본은 jq가 없거나 해석이 실패하면 **exit 2**로 막고, grep 결과가 **1(불일치)일 때만** 통과시킵니다. 0(일치)과 2 이상(검사기 오류)은 모두 차단합니다. 이것이 **fail-closed**입니다.



● 입력 ● 통과 ● 차단 ● 검사 실패

/opt/dlp/dlp-guard.sh

수정본 요약 (Lab 2 TASK 02)

```
#!/bin/bash
INPUT=$(cat)
command -v jq >/dev/null || { echo "DLP: 검사기 오류(jq 없음)"
>&2; exit 2; }
TEXT=$(echo "$INPUT" | jq -r '.prompt // (.tool_input |
tostring)') \
  || { echo "DLP: 입력 해석 실패" >&2; exit 2; }
PAT='...'    # 위와 같음
echo "$TEXT" | grep -Eq "$PAT"; RC=$?
[ "$RC" -eq 1 ] && exit 0    # 1 = 패턴 없음, 통과
echo "DLP: 차단 (grep $RC)" >&2    # 0 일치, 2+ 검사기 오류
exit 2
```

if grep ...; then 형태는 grep 오류를 불일치처럼 통과시킵니다.

- jq 없음, 해석 실패: exit 2
- grep 1(불일치)만 통과
- 스크립트 자체가 없으면: onFailure: block (다음 장)

‹ 5 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

jq 없이 exit 127이면?

QUICK CHECK, 손을 들어 답해 주세요

DLP 훅이 jq 미설치로 exit 127로 끝나면 어떻게 될까요?

`exit 0` → 진행

`exit 2` → **차단**

`exit 127` → 비차단 오류, 진행

A 차단된다

B 통과된다

C 세션이 종료된다

D 자동 재시도

정답 B. exit 2만 차단입니다. 그 외 0이 아닌 종료는 비차단 오류로 처리되어 그대로 진행됩니다. onFailure: "block"을 걸면 이 경우도 차단됩니다.

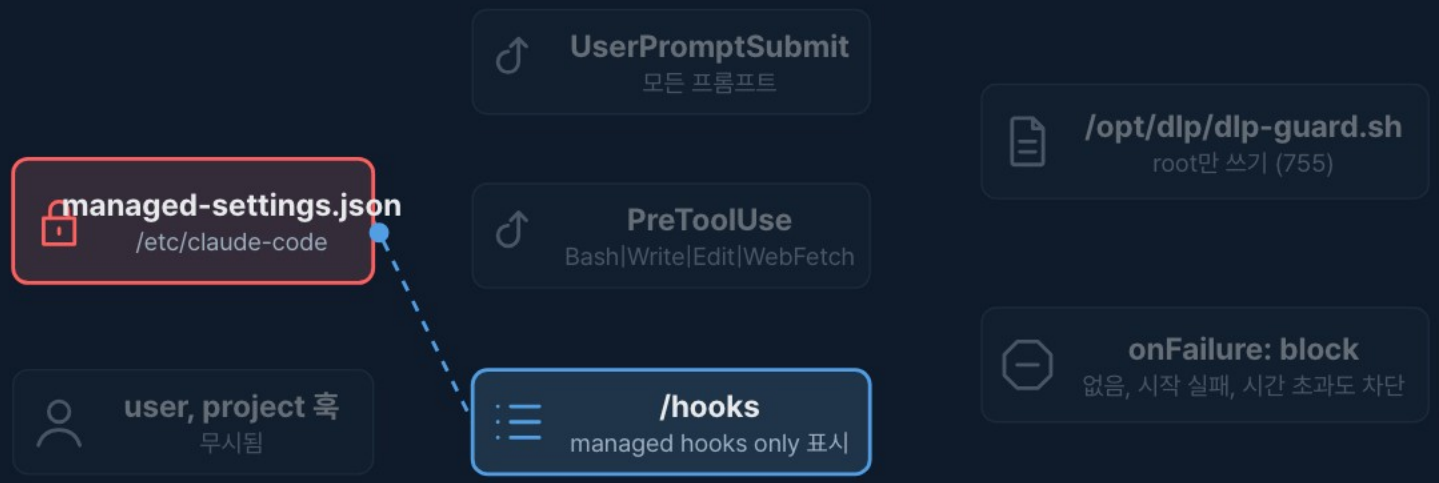
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

정책으로 강제하는 훅

사용자가 끄지 못하게, 고장 나도 막히게

STEP 5 **/hooks로 확인합니다**

세션에서 **/hooks**를 열면 **managed hooks only** 표시와 "Only hooks from managed settings run here"가 보입니다. managed 훅 자체는 이 목록에 나오지 않습니다.



● 정책, 강제 ● 훅 지점 ● 스크립트 ● 무시됨

확인

Claude 입력창

```
/hooks
# managed hooks only
# Only hooks from managed settings run here
```

랩 TASK 01 마지막 단계에서 직접 봅니다.

- managed hooks only 표시 확인
- managed 훅은 목록에 안 보임
- managed 설정이 있으면 mod가 승인해도 훅 차단은 유지

‹ 6 / 6 ›

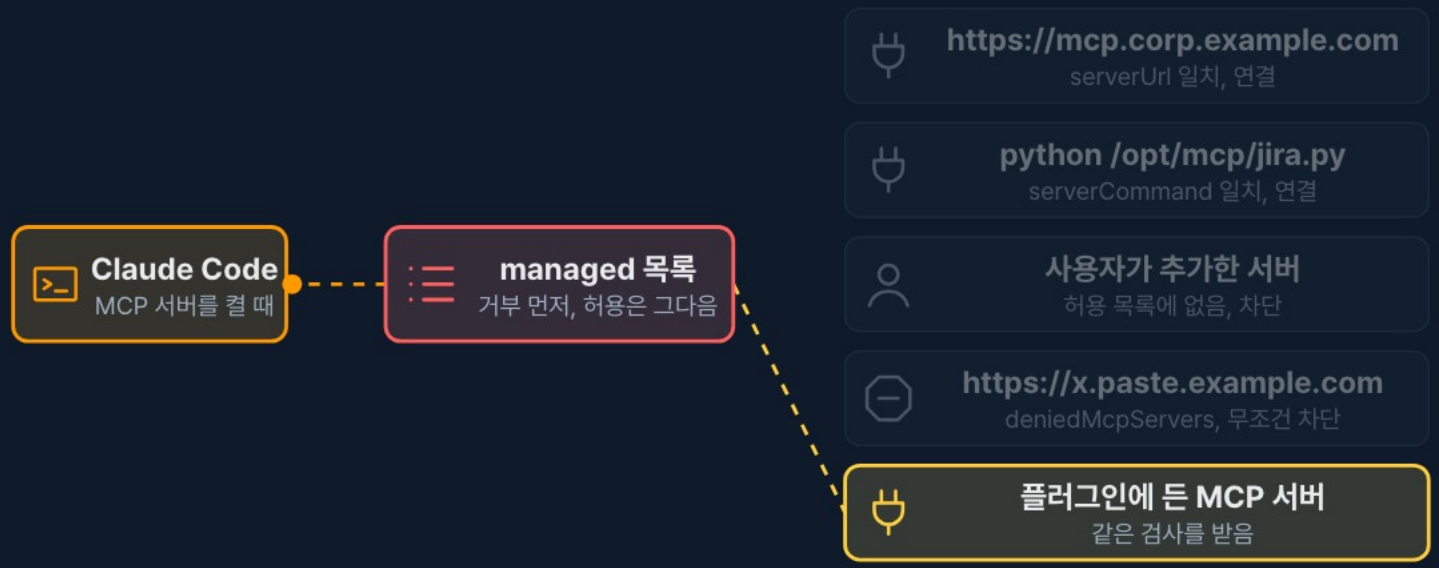
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

MCP 서버 하나가 통로 하나

새 반출 채널을 정책으로 닫는다

STEP 4 **플러그인에 든 MCP도 같은 검사**

플러그인이 가져온 MCP 서버도 **같은 허용, 거부 목록**을 지나야 켜집니다. 플러그인으로 우회할 수 없습니다. 서버 목록을 통째로 고정하려면 **managed-mcp.json**을 씁니다.



● 허용, 연결 ● 거부 ● 목록에 없음 ● 플러그인 경로

managed-settings.json (MCP)

```
"allowManagedMcpServersOnly": true,
"allowedMcpServers": [
  { "serverUrl":
      "https://mcp.corp.example.com/*" },
  { "serverCommand":
      ["python", "/opt/mcp/jira.py"] }
],
"deniedMcpServers": [
  { "serverUrl":
      "https://*.paste.example.com/*" }
]
```

플러그인 자체의 허용 목록은 **Ch4**에서 다룹니다.

- 플러그인, claude.ai 커넥터도 검사
- managed-mcp.json: 정해 둔 서버만, 나머지 금지
- 사용 현황은 OTel로 (OTEL_LOG_TOOL_DETAILS=1)

‹ 5 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

감사 근거가 남는 네 곳

요청 하나가 어디에 무엇을 남기는가

4 AWS invocation logging: 원문까지, 선택

Bedrock **model invocation logging**을 켜면 프롬프트와 응답 **원문**이 S3나 CloudWatch Logs에 남습니다. 그 저장소 자체가 **민감 데이터**가 됩니다.



Bedrock 설정 (요약)

`aws bedrock put-model-invocation-logging-configuration`

```
--logging-config '{
  "s3Config": { "bucketName": "fsi-bedrock-logs" },
  "textDataDeliveryEnabled": true
}'
```

Anthropic으로 가는 텔레메트리는 Bedrock에서 **기본 꺼짐**, WebFetch 사전 검사만 호스트명을 보냅니다.

- 기본 꺼짐, 계정과 리전 단위로 켬
- S3, CloudWatch Logs로
- 저장소 접근 권한, 암호화, 보존 기간 먼저

● PC 기록 ● 사내 수집기 ● AWS 기본 기록 ● 선택, 원문

‹ 5 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

더 깊은 내용은 실습에서 손으로

강의에서 줄인 주제와 실습 TASK

Lab 1 TASK 01	지금의 연결 읽기: env, 추론 프로필, 최소 IAM 정책과 비교	IAM 세부
Lab 1 TASK 02부터 04	정책 배포, 개인 설정으로 우회 시도, 정책 파일이 깨지면?	정책 배포
Lab 1 TASK 06	같은 정책을 게이트웨이 그룹 정책으로 내려보내기	게이트웨이
Lab 2 TASK 01, 02	DLP 훅을 정책으로 강제하고, fail-open을 고치기	훅
Lab 2 TASK 03	auto mode로 확인 창을 줄이고 경계 긋기	auto mode
Lab 2 선택 TASK	샌드박스로 curl 빈틈 막기, Bedrock Guardrails, 확장 통제	DLP 층

망분리 준비물, OS별 차이, 쿼터는 PDF 강의자료(정책편, 가드레일편)의 더 알아보기에 있습니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

기억할 여섯 줄

Chapter 2 Key Takeaways

01

전용선과 PrivateLink면 인터넷 없이 Bedrock에, 연결은 USE_BEDROCK과 리전

02

장기 키 대신 SSO 프로필 + awsAuthRefresh

03

모델은 반드시 고정, 팀마다 다른 이름표(추론 프로필)로 누구의 호출인지

04

정책은 managed-settings.json, 충돌하면 managed가 이기고 /status로 확인

05

요청은 훅, 권한 규칙, 분류기, 샌드박스를 차례로 지남, 훅은 exit 2만 차단

06

DLP는 프롬프트 앞에서, 못 막은 것은 기록으로

비용을 줄이는 법은 Ch5(개인), 조직이 비용을 보고 통제하는 법은 Ch6에서 이어집니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank You

다음은 점심입니다. 식사 후 통합 랩에서 Preflight와 Ch1 Lab을 마친 뒤, 오늘 본 정책과 훅을 Ch2 Lab 1, Lab 2에서 직접 배포하고 뚫어 봅니다. 질문은 Slack #20261116-claude-code-workshop-for-fsi 채널에서 언제든 받습니다.

← 목차로 돌아가기 Chapter 3 →

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.